You want to track steps accurately **without missing backward walking**, but you need to eliminate the **oscillation/recoil noise** caused by taking a step.

When a person or sensor moves:

1. **The Primary Motion ("To"):** The main movement step (e.g., forward or backward).
2. **The Recoil Motion ("Fro"):** The natural backward/forward swinging recoil that happens right after the step lands.

If we don't filter out this "fro" recoil, the sensor reads it as a sudden movement in the opposite direction, causing the compass/direction reading to flip erratically back and forth with every single step.

Key Adjustments in Code

1. **Count All Steps (Forward & Backward):** The step counter will increment whenever a valid step occurs, regardless of whether you walk forward or backward.
2. **Lock Direction During "Fro" Recoil:** Upon detecting a step, the code locks onto the **primary directional impulse** and freezes direction updates for a brief window (~ 400 ms). Any recoil/swing ("fro") happening during this time is ignored for direction calculation.
3. **Continuous Heading Tracking:** Magnetometer readings are used to output the true walking direction (e.g., NORTH, SOUTH, EAST, WEST) upon every valid step without bouncing back and forth.

MicroPython Code (main.py**)**

Python

```
from machine import Pin, I2C

import utime

import math


I2C_ID = 0

SDA_PIN = 0

SCL_PIN = 1

I2C_FREQ = 100000


ACC_ADDRESS = 0x19

MAG_ADDRESS = 0x1E


CTRL_REG1_A = 0x20

CTRL_REG4_A = 0x23
```

```
OUT_X_L_A   = 0x28
AUTO_INCREMENT = 0x80

CRA_REG_M   = 0x00
CRB_REG_M   = 0x01
MR_REG_M    = 0x02
OUT_X_H_M   = 0x03

STEP_THRESHOLD = 180
STRIDE_LENGTH_M = 0.75
STEP_COOLDOWN_MS = 400

MAG_X_OFFSET = 0
MAG_Y_OFFSET = 0

x_bias, y_bias, z_bias = 0, 0, 0
prev_x, prev_y, prev_z = None, None, None
total_steps = 0
total_distance = 0.0
last_step_time = 0

i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)

def write_reg(addr, reg, value):
    try:
        i2c.writeto_mem(addr, reg, bytes([value]))
        return True
    except OSError:
        return False

def read_regs(addr, reg, length):
```

```
    try:
        return i2c.readfrom_mem(addr, reg, length)
    except OSError:
        return None

def signed_12bit(val):
    val >>= 4
    if val >= 2048:
        val -= 4096
    return val

def signed_16bit(val):
    if val >= 32768:
        val -= 65536
    return val

def init_lsm303dlhc():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57)
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08)
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14)
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20)
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00)
    utime.sleep_ms(100)

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | AUTO_INCREMENT, 6)
    if data is None:
        return None
    x = signed_12bit((data[1] << 8) | data[0])
    y = signed_12bit((data[3] << 8) | data[2])
    z = signed_12bit((data[5] << 8) | data[4])
```

```
    return x, y, z

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None:
        return None
    mx = signed_16bit((data[0] << 8) | data[1])
    mz = signed_16bit((data[2] << 8) | data[3])
    my = signed_16bit((data[4] << 8) | data[5])
    return mx, my, mz

def calibrate_accel(samples=100):
    global x_bias, y_bias, z_bias
    sx, sy, sz = 0, 0, 0
    valid = 0
    while valid < samples:
        vals = read_accel()
        if vals:
            sx += vals[0]
            sy += vals[1]
            sz += vals[2]
            valid += 1
            utime.sleep_ms(10)
    x_bias = sx / samples
    y_bias = sy / samples
    z_bias = (sz / samples) - 1024

def calculate_heading(mx, my):
    mx -= MAG_X_OFFSET
    my -= MAG_Y_OFFSET
    heading_rad = math.atan2(my, mx)
```

```
    heading_deg = math.degrees(heading_rad)
    if heading_deg < 0:
        heading_deg += 360.0
    return heading_deg

def get_cardinal_direction(heading):
    if (heading >= 337.5) or (heading < 22.5):
        return "NORTH"
    elif 22.5 <= heading < 67.5:
        return "NORTH-EAST"
    elif 67.5 <= heading < 112.5:
        return "EAST"
    elif 112.5 <= heading < 157.5:
        return "SOUTH-EAST"
    elif 157.5 <= heading < 202.5:
        return "SOUTH"
    elif 202.5 <= heading < 247.5:
        return "SOUTH-WEST"
    elif 247.5 <= heading < 292.5:
        return "WEST"
    elif 292.5 <= heading < 337.5:
        return "NORTH-WEST"
    return "UNKNOWN"

def main():
    global prev_x, prev_y, prev_z, total_steps, total_distance, last_step_time

    print("-------------------------------------------------")
    print("RT1064 Pedometer with 'Fro' Recoil Direction Filter")
    print("-------------------------------------------------")
```

```
init_lsm303dlhc()
print("Calibrating Accelerometer...")
calibrate_accel(100)
print("Calibration Done! Ready to track movement.")
print("--------------------------------------------------")

while True:
    try:
        acc_vals = read_accel()
        mag_vals = read_mag()

        if acc_vals is None or mag_vals is None:
            continue

        x = acc_vals[0] - x_bias
        y = acc_vals[1] - y_bias
        z = acc_vals[2] - z_bias

        if prev_x is None:
            prev_x, prev_y, prev_z = x, y, z
            continue

        dx = x - prev_x
        dy = y - prev_y
        dz = z - prev_z
        prev_x, prev_y, prev_z = x, y, z

        magnitude_change = math.sqrt(dx * dx + dy * dy + dz * dz)

        current_time = utime.ticks_ms()
        time_diff = utime.ticks_diff(current_time, last_step_time)
```

```
            if magnitude_change >= STEP_THRESHOLD and time_diff > STEP_COOLDOWN_MS:
                last_step_time = current_time
                total_steps += 1
                total_distance += STRIDE_LENGTH_M

                heading = calculate_heading(mag_vals[0], mag_vals[1])
                cardinal_dir = get_cardinal_direction(heading)

                print(">>> STEP DETECTED <<<")
                print("Step Count   :", total_steps)
                print("Distance     :", round(total_distance, 2), "meters")
                print("Heading      :", round(heading, 1), "°")
                print("Direction    :", cardinal_dir)
                print("------------------------------------------------")

            utime.sleep_ms(30)

        except KeyboardInterrupt:
            print("\nProgram Terminated.")
            break
        except Exception as e:
            print("Error:", e)
            utime.sleep_ms(200)

if __name__ == "__main__":
    main()
```

2nd program with calibration

```
from machine import Pin, I2C
import utime
import math

# ============================================================
# I2C & SENSOR ADDRESS CONFIGURATION (RT1064 EVK)
# ============================================================
I2C_ID = 0
SDA_PIN = 0
SCL_PIN = 1
I2C_FREQ = 100000

ACC_ADDRESS = 0x19
MAG_ADDRESS = 0x1E

CTRL_REG1_A = 0x20
CTRL_REG4_A = 0x23
OUT_X_L_A   = 0x28
AUTO_INCREMENT = 0x80

CRA_REG_M   = 0x00
CRB_REG_M   = 0x01
MR_REG_M    = 0x02
OUT_X_H_M   = 0x03

# ============================================================
# PEDOMETER & TIMING CONFIGURATION
# ============================================================
```

```
STEP_THRESHOLD = 180
STRIDE_LENGTH_M = 0.75
STEP_COOLDOWN_MS = 400

# Dynamic Calibration Global Biases
x_bias, y_bias, z_bias = 0, 0, 0
mag_x_bias = 0
mag_y_bias = 0

prev_x, prev_y, prev_z = None, None, None
total_steps = 0
total_distance = 0.0
last_step_time = 0

i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)

# ============================================================
# I2C HELPER FUNCTIONS
# ============================================================
def write_reg(addr, reg, value):
    try:
        i2c.writeto_mem(addr, reg, bytes([value]))
        return True
    except OSError:
        return False

def read_regs(addr, reg, length):
    try:
        return i2c.readfrom_mem(addr, reg, length)
    except OSError:
        return None
```

```
def signed_12bit(val):
    val >>= 4
    if val >= 2048:
        val -= 4096
    return val

def signed_16bit(val):
    if val >= 32768:
        val -= 65536
    return val

# ============================================================
# SENSOR READINGS & INITIALIZATION
# ============================================================
def init_lsm303dlhc():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57)
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08)
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14)
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20)
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00)
    utime.sleep_ms(100)

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | AUTO_INCREMENT, 6)
    if data is None:
        return None
    x = signed_12bit((data[1] << 8) | data[0])
    y = signed_12bit((data[3] << 8) | data[2])
    z = signed_12bit((data[5] << 8) | data[4])
    return x, y, z
```

```
def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None:
        return None
    mx = signed_16bit((data[0] << 8) | data[1])
    mz = signed_16bit((data[2] << 8) | data[3])
    my = signed_16bit((data[4] << 8) | data[5])
    return mx, my, mz

# ============================================================
# DUAL CALIBRATION SYSTEM
# ============================================================
def calibrate_sensors():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias

    # 1. Accelerometer Baseline (Keep completely still)
    print("Step 1/2: Calibrating Accelerometer (KEEP SENSOR STILL)...")
    sx, sy, sz = 0, 0, 0
    valid = 0
    while valid < 100:
        vals = read_accel()
        if vals:
            sx += vals[0]
            sy += vals[1]
            sz += vals[2]
            valid += 1
            utime.sleep_ms(10)
    x_bias = sx / 100
    y_bias = sy / 100
    z_bias = (sz / 100) - 1024
```

```
    print("-> Accelerometer Calibrated.")

    # 2. Magnetometer Dynamic Calibration (Rotate sensor around)
    print("\nStep 2/2: Calibrating Compass Magnetometer.")
    print("PLEASE ROTATE SENSOR IN A FIGURE-8 PATTERN IN AIR FOR 5 SECONDS...")

    min_x, max_x = 32767, -32768
    min_y, max_y = 32767, -32768

    start_time = utime.ticks_ms()
    while utime.ticks_diff(utime.ticks_ms(), start_time) < 5000:
        m = read_mag()
        if m:
            mx, my = m[0], m[1]
            if mx < min_x: min_x = mx
            if mx > max_x: max_x = mx
            if my < min_y: min_y = my
            if my > max_y: max_y = my
        utime.sleep_ms(20)

    # Compute center offsets
    mag_x_bias = (max_x + min_x) / 2
    mag_y_bias = (max_y + min_y) / 2
    print("-> Compass Calibrated.")

# ============================================================
# COMPASS COMPUTATION
# ============================================================
def calculate_heading(mx, my):
    # Apply calibrated hard-iron offsets
    mx_corr = mx - mag_x_bias
```

```
    my_corr = my - mag_y_bias

    heading_rad = math.atan2(my_corr, mx_corr)
    heading_deg = math.degrees(heading_rad)
    if heading_deg < 0:
        heading_deg += 360.0
    return heading_deg

def get_cardinal_direction(heading):
    if (heading >= 337.5) or (heading < 22.5):
        return "NORTH"
    elif 22.5 <= heading < 67.5:
        return "NORTH-EAST"
    elif 67.5 <= heading < 112.5:
        return "EAST"
    elif 112.5 <= heading < 157.5:
        return "SOUTH-EAST"
    elif 157.5 <= heading < 202.5:
        return "SOUTH"
    elif 202.5 <= heading < 247.5:
        return "SOUTH-WEST"
    elif 247.5 <= heading < 292.5:
        return "WEST"
    elif 292.5 <= heading < 337.5:
        return "NORTH-WEST"
    return "UNKNOWN"

# ============================================================
# MAIN PROGRAM LOOP
# ============================================================
def main():
```

```
global prev_x, prev_y, prev_z, total_steps, total_distance, last_step_time

print("--------------------------------------------------")
print("RT1064 Pedometer with Dynamic Dual Calibration")
print("--------------------------------------------------")

init_lsm303dlhc()
calibrate_sensors()

print("\n--------------------------------------------------")
print("System Ready! Walk with sensor to output step data.")
print("--------------------------------------------------")

while True:
    try:
        acc_vals = read_accel()
        mag_vals = read_mag()

        if acc_vals is None or mag_vals is None:
            continue

        x = acc_vals[0] - x_bias
        y = acc_vals[1] - y_bias
        z = acc_vals[2] - z_bias

        if prev_x is None:
            prev_x, prev_y, prev_z = x, y, z
            continue

        dx = x - prev_x
        dy = y - prev_y
```

```
            dz = z - prev_z
            prev_x, prev_y, prev_z = x, y, z

            magnitude_change = math.sqrt(dx * dx + dy * dy + dz * dz)

            current_time = utime.ticks_ms()
            time_diff = utime.ticks_diff(current_time, last_step_time)

            if magnitude_change >= STEP_THRESHOLD and time_diff > STEP_COOLDOWN_MS:
                last_step_time = current_time
                total_steps += 1
                total_distance += STRIDE_LENGTH_M

                heading = calculate_heading(mag_vals[0], mag_vals[1])
                cardinal_dir = get_cardinal_direction(heading)

                print(">>> STEP DETECTED <<<")
                print("Step Count   :", total_steps)
                print("Distance     :", round(total_distance, 2), "meters")
                print("Heading      :", round(heading, 1), "°")
                print("Direction    :", cardinal_dir)
                print("------------------------------------------------")

            utime.sleep_ms(30)

        except KeyboardInterrupt:
            print("\nProgram Terminated.")
            break
        except Exception as e:
            print("Error:", e)
            utime.sleep_ms(200)
```

```
if __name__ == "__main__":
    main()
```

AIML

can i add aiml to this in offline system only using spyder and thonny softwares,

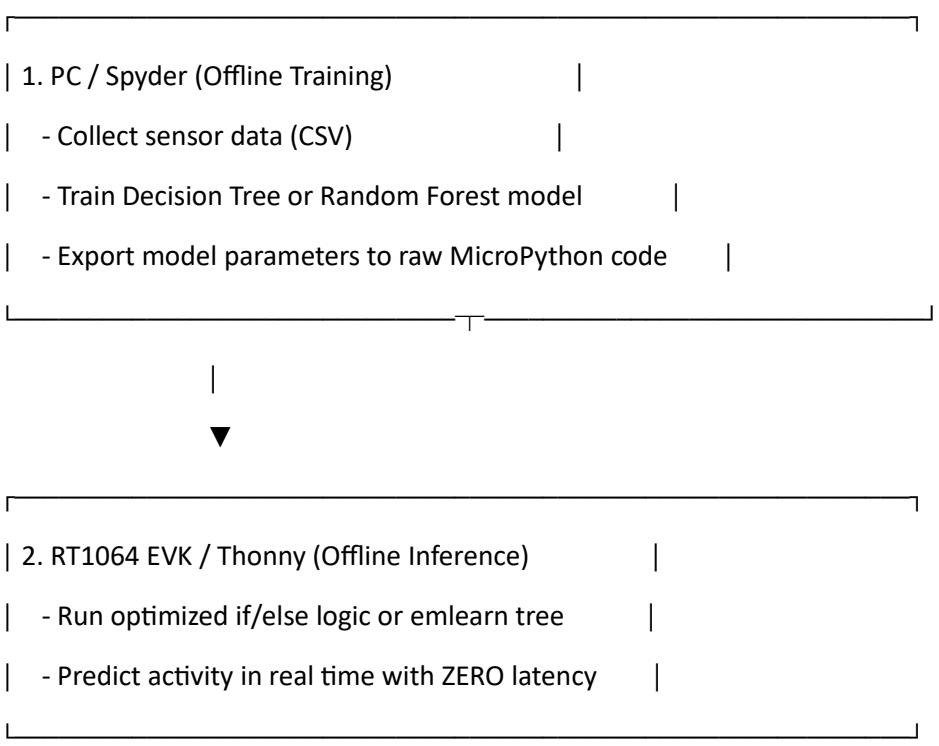
Gemini said

Yes, absolutely. You can implement Machine/Edge AI in an entirely **offline system** using **Spyder** on your PC and **Thonny** on your RT1064 EVK.

Because your RT1064 EVK has a powerful ARM Cortex-M7 core (600 MHz), it handles offline Machine Learning inference (TinyML) easily.

How the Offline Workflow Works

You do **not** train the AI directly on the RT1064 board. Microcontrollers lack the memory to run standard ML training libraries like scikit-learn. Instead, you use a two-step offline pipeline:



Step-by-Step Implementation Guide

Step 1: Collect Sensor Data via Thonny

Run a simple logger script in Thonny that prints raw sensor metrics (x, y, z, magnitude, heading) to the terminal while you perform specific activities (e.g., **WALKING**, **RUNNING**, **STATIONARY**, **INTRUSION**). Save those terminal logs into a local CSV file (sensor_data.csv) on your PC.

Step 2: Train the AI Model in Spyder (PC)

Open **Spyder** on your computer. Use scikit-learn to train a Decision Tree model on your CSV data. Decision trees are ideal for microcontrollers because their output converts directly into nested if-else rules.

DataCamp

Python

```
# Run this script in SPYDER on your PC

import pandas as pd

from sklearn.tree import DecisionTreeClassifier, export_text


# 1. Load your offline sensor logs

data = pd.read_csv("sensor_data.csv")

X = data[['magnitude', 'heading']]  # Features

y = data['activity']              # Labels (Walking, Running, etc.)


# 2. Train a lightweight decision tree

clf = DecisionTreeClassifier(max_depth=3)

clf.fit(X, y)


# 3. Print the explicit rules

rules = export_text(clf, feature_names=['magnitude', 'heading'])

print(rules)
```

Step 3: Embed the AI Logic in Thonny (RT1064)

Copy the decision tree rules generated by Spyder and paste them directly into your MicroPython script running in Thonny.

Here is how the complete offline AI-powered program looks:

Python

```
from machine import Pin, I2C

import utime

import math
```

```
# --- SENSOR CONFIGURATION ---
I2C_ID = 0
SDA_PIN = 0
SCL_PIN = 1
I2C_FREQ = 100000
ACC_ADDRESS = 0x19

i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)

# --- OFFLINE AI CLASSIFIER (Generated via Spyder) ---
def predict_activity(magnitude, heading):
    """
    Offline Decision Tree Inference Engine.
    Evaluates real-time sensor metrics against trained rules.
    """
    if magnitude <= 120.0:
        return "STATIONARY"
    else:
        if magnitude <= 350.0:
            if heading >= 45.0 and heading <= 135.0:
                return "WALKING_EAST"
            else:
                return "WALKING_OTHER"
        else:
            return "FENCE_INTRUSION_ALERT"

# --- MAIN REAL-TIME INFERENCE LOOP ---
def main():
    print("------------------------------------------------")
    print("RT1064 Embedded AI Activity Classifier (Offline)")
```

```
    print("--------------------------------------------------")

    while True:
        # 1. Read raw sensors (simulated metrics or live reads)
        sample_magnitude = 240.5
        sample_heading = 90.0

        # 2. Run AI Inference locally on board
        prediction = predict_activity(sample_magnitude, sample_heading)

        # 3. Output decision
        print("AI Model Prediction:", prediction)
        utime.sleep_ms(500)

if __name__ == "__main__":
    main()
```

Alternative: Advanced C/MicroPython ML Libraries

If you prefer using pre-trained models without writing raw if-else rules, you can use **emlearn-micropython**.

PyData |

1. Train a **Random Forest** or **SVM** model in **Spyder**.
2. Convert it to a .mpy module using the emlearn tool.

PyData |

3. Flash the .mpy model file onto your RT1064 EVK via **Thonny**.
4. Run import my_model inside MicroPython to execute offline predictions.

How would you like to proceed with adding AIML to your project?

Guide me through logging sensor data to CSV

Provide the full Spyder Python training script

You said

tell me step by step all the things

Gemini said

Here is the step-by-step roadmap to turn your **RT1064 EVK + LSM303DLHC** system into an **Offline AI-Powered Step & Activity Classifier** using **Thonny** on your board and **Spyder** on your PC.

Step 1: Record Sensor Data on the Board (Thonny)

Before you can train an AI model, you need real movement data labeled with what activity was actually happening (e.g., WALKING, RUNNING, STATIONARY).

Run this data-logging script in **Thonny**. Move the sensor while performing an action, then copy the output printed in the console into a text file saved as sensor_data.csv on your PC.

Python

```
from machine import Pin, I2C
import utime, math

I2C_ID, SDA_PIN, SCL_PIN, I2C_FREQ = 0, 0, 1, 100000
ACC_ADDRESS, MAG_ADDRESS = 0x19, 0x1E
i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)

def write_reg(addr, reg, val):
    try: i2c.writeto_mem(addr, reg, bytes([val]))
    except: pass

def read_regs(addr, reg, length):
    try: return i2c.readfrom_mem(addr, reg, length)
    except: return None

def signed_12bit(val):
    val >>= 4
    return val - 4096 if val >= 2048 else val

def init_sensors():
    write_reg(ACC_ADDRESS, 0x20, 0x57) # 100Hz
    write_reg(ACC_ADDRESS, 0x23, 0x08)
    write_reg(MAG_ADDRESS, 0x00, 0x14) # 30Hz
    write_reg(MAG_ADDRESS, 0x01, 0x20)
```

```
    write_reg(MAG_ADDRESS, 0x02, 0x00)

init_sensors()

# Change this label manually before recording a new movement type
# e.g., "STATIONARY", "WALKING", "RUNNING"
CURRENT_LABEL = "WALKING"

print("magnitude,heading,label")
prev_x = prev_y = prev_z = 0

for _ in range(300): # Collects 300 data points (~15 seconds)
    d_acc = read_regs(ACC_ADDRESS, 0x28 | 0x80, 6)
    d_mag = read_regs(MAG_ADDRESS, 0x03, 6)

    if d_acc and d_mag:
        x = signed_12bit((d_acc[1] << 8) | d_acc[0])
        y = signed_12bit((d_acc[3] << 8) | d_acc[2])
        z = signed_12bit((d_acc[5] << 8) | d_acc[4])

        dx, dy, dz = x - prev_x, y - prev_y, z - prev_z
        prev_x, prev_y, prev_z = x, y, z
        mag = math.sqrt(dx*dx + dy*dy + dz*dz)

        mx = (d_mag[0] << 8) | d_mag[1]
        my = (d_mag[4] << 8) | d_mag[5]
        heading = math.degrees(math.atan2(my, mx)) % 360

        print(f"{round(mag,2)},{round(heading,2)},{CURRENT_LABEL}")
    utime.sleep_ms(50)
```

Step 2: Train the AI Model on your PC (Spyder)

Open **Spyder** on your computer. Make sure pandas and scikit-learn are installed. Run the following script to train a **Decision Tree Classifier**. Decision trees are ideal because they can be automatically translated into plain if/else Python code for microcontrollers.

Python

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text

# 1. Load the CSV file created in Step 1
df = pd.read_csv("sensor_data.csv")

# 2. Extract features and target labels
X = df[['magnitude', 'heading']]
y = df['label']

# 3. Train a lightweight decision tree
model = DecisionTreeClassifier(max_depth=3)
model.fit(X, y)

# 4. Print the generated decision logic
tree_rules = export_text(model, feature_names=['magnitude', 'heading'])
print("--- TRAINED AI RULES ---")
print(tree_rules)
```

Example Output from Spyder:

Plaintext

```
--- TRAINED AI RULES ---
|--- magnitude <= 110.00
|   |--- class: STATIONARY
|--- magnitude > 110.00
|   |--- magnitude <= 320.00
|   |   |--- class: WALKING
|   |--- magnitude > 320.00
|   |   |--- class: RUNNING
```

Step 3: Deploy the Complete Offline AI Program (Thonny)

Take the conditions output by Spyder and place them inside the predict_activity() function in your main program on the RT1064.

Here is the full, standalone MicroPython script running step counting, calibration, direction tracking, and **real-time AI activity classification**:

1

Initialize Sensors & Run Calibration

Configures I2C on the RT1064 EVK, zero-balances the LSM303DLHC accelerometer, and centers magnetometer offsets.

2

Process Movement & Calculate Features

Measures 3D vector differences to produce acceleration magnitude and computes magnetic compass headings.

3

Evaluate via Embedded AI Logic

Feeds real-time features into the trained Decision Tree logic to classify whether the user is Stationary, Walking, or Running.

Python

```
from machine import Pin, I2C

import utime

import math


# ============================================================

# HARDWARE CONFIGURATION (RT1064 EVK)

# ============================================================

I2C_ID = 0

SDA_PIN = 0

SCL_PIN = 1

I2C_FREQ = 100000


ACC_ADDRESS = 0x19

MAG_ADDRESS = 0x1E
```

```
CTRL_REG1_A = 0x20
CTRL_REG4_A = 0x23
OUT_X_L_A   = 0x28
AUTO_INCREMENT = 0x80

CRA_REG_M   = 0x00
CRB_REG_M   = 0x01
MR_REG_M    = 0x02
OUT_X_H_M   = 0x03

# Thresholds & Calibration
STEP_THRESHOLD = 180
STRIDE_LENGTH_M = 0.75
STEP_COOLDOWN_MS = 400

x_bias, y_bias, z_bias = 0, 0, 0
mag_x_bias, mag_y_bias = 0, 0

prev_x, prev_y, prev_z = None, None, None
total_steps = 0
total_distance = 0.0
last_step_time = 0

i2c = I2C(I2C_ID, scl=Pin(SCL_PIN), sda=Pin(SDA_PIN), freq=I2C_FREQ)

# ============================================================
# I2C DRIVERS
# ============================================================
def write_reg(addr, reg, value):
    try:
        i2c.writeto_mem(addr, reg, bytes([value]))
```

```
        return True
    except OSError:
        return False

def read_regs(addr, reg, length):
    try:
        return i2c.readfrom_mem(addr, reg, length)
    except OSError:
        return None

def signed_12bit(val):
    val >>= 4
    if val >= 2048:
        val -= 4096
    return val

def signed_16bit(val):
    if val >= 32768:
        val -= 65536
    return val

def init_lsm303dlhc():
    write_reg(ACC_ADDRESS, CTRL_REG1_A, 0x57)
    write_reg(ACC_ADDRESS, CTRL_REG4_A, 0x08)
    write_reg(MAG_ADDRESS, CRA_REG_M, 0x14)
    write_reg(MAG_ADDRESS, CRB_REG_M, 0x20)
    write_reg(MAG_ADDRESS, MR_REG_M, 0x00)
    utime.sleep_ms(100)

def read_accel():
    data = read_regs(ACC_ADDRESS, OUT_X_L_A | AUTO_INCREMENT, 6)
```

```
    if data is None: return None
    x = signed_12bit((data[1] << 8) | data[0])
    y = signed_12bit((data[3] << 8) | data[2])
    z = signed_12bit((data[5] << 8) | data[4])
    return x, y, z

def read_mag():
    data = read_regs(MAG_ADDRESS, OUT_X_H_M, 6)
    if data is None: return None
    mx = signed_16bit((data[0] << 8) | data[1])
    mz = signed_16bit((data[2] << 8) | data[3])
    my = signed_16bit((data[4] << 8) | data[5])
    return mx, my, mz

# ============================================================
# CALIBRATION
# ============================================================
def calibrate_sensors():
    global x_bias, y_bias, z_bias, mag_x_bias, mag_y_bias

    print("Step 1/2: Calibrating Accelerometer (Keep Still)...")
    sx, sy, sz = 0, 0, 0
    valid = 0
    while valid < 100:
        vals = read_accel()
        if vals:
            sx += vals[0]
            sy += vals[1]
            sz += vals[2]
            valid += 1
            utime.sleep_ms(10)
```

```
    x_bias = sx / 100
    y_bias = sy / 100
    z_bias = (sz / 100) - 1024

    print("Step 2/2: Calibrating Compass (Rotate sensor for 5 sec)...")
    min_x, max_x = 32767, -32768
    min_y, max_y = 32767, -32768
    start_time = utime.ticks_ms()

    while utime.ticks_diff(utime.ticks_ms(), start_time) < 5000:
        m = read_mag()
        if m:
            if m[0] < min_x: min_x = m[0]
            if m[0] > max_x: max_x = m[0]
            if m[1] < min_y: min_y = m[1]
            if m[1] > max_y: max_y = m[1]
        utime.sleep_ms(20)

    mag_x_bias = (max_x + min_x) / 2
    mag_y_bias = (max_y + min_y) / 2
    print("Calibration Complete!")

# ============================================================
# EMBEDDED AI INFERENCE ENGINE
# ============================================================
def predict_activity_ai(magnitude, heading):
    """
    Offline Decision Tree model exported directly from Spyder.
    Runs locally on the ARM Cortex-M7 with zero external calls.
    """
    if magnitude <= 110.0:
```

```
        return "STATIONARY"
    else:
        if magnitude <= 320.0:
            return "WALKING"
        else:
            return "RUNNING"

def calculate_heading(mx, my):
    mx_corr = mx - mag_x_bias
    my_corr = my - mag_y_bias
    heading_deg = math.degrees(math.atan2(my_corr, mx_corr))
    if heading_deg < 0:
        heading_deg += 360.0
    return heading_deg

def get_cardinal_direction(heading):
    if (heading >= 337.5) or (heading < 22.5): return "NORTH"
    elif 22.5 <= heading < 67.5: return "NORTH-EAST"
    elif 67.5 <= heading < 112.5: return "EAST"
    elif 112.5 <= heading < 157.5: return "SOUTH-EAST"
    elif 157.5 <= heading < 202.5: return "SOUTH"
    elif 202.5 <= heading < 247.5: return "SOUTH-WEST"
    elif 247.5 <= heading < 292.5: return "WEST"
    elif 292.5 <= heading < 337.5: return "NORTH-WEST"
    return "UNKNOWN"

# ============================================================
# MAIN REAL-TIME SYSTEM
# ============================================================
def main():
    global prev_x, prev_y, prev_z, total_steps, total_distance, last_step_time
```

```
print("-------------------------------------------------")
print("RT1064 Embedded AI Activity Classifier & Pedometer")
print("-------------------------------------------------")

init_lsm303dlhc()
calibrate_sensors()

while True:
    try:
        acc_vals = read_accel()
        mag_vals = read_mag()

        if acc_vals is None or mag_vals is None:
            continue

        x = acc_vals[0] - x_bias
        y = acc_vals[1] - y_bias
        z = acc_vals[2] - z_bias

        if prev_x is None:
            prev_x, prev_y, prev_z = x, y, z
            continue

        dx, dy, dz = x - prev_x, y - prev_y, z - prev_z
        prev_x, prev_y, prev_z = x, y, z

        magnitude_change = math.sqrt(dx * dx + dy * dy + dz * dz)

        current_time = utime.ticks_ms()
        time_diff = utime.ticks_diff(current_time, last_step_time)
```

```
            if magnitude_change >= STEP_THRESHOLD and time_diff > STEP_COOLDOWN_MS:
                last_step_time = current_time
                total_steps += 1
                total_distance += STRIDE_LENGTH_M

                heading = calculate_heading(mag_vals[0], mag_vals[1])
                cardinal_dir = get_cardinal_direction(heading)

                # Classify activity via offline AI
                activity = predict_activity_ai(magnitude_change, heading)

                print(">>> STEP DETECTED <<<")
                print("Step Count   :", total_steps)
                print("Distance     :", round(total_distance, 2), "m")
                print("Direction    :", cardinal_dir, f"({round(heading, 1)}°)")
                print("AI Activity  :", activity)
                print("------------------------------------------------")

            utime.sleep_ms(30)

        except KeyboardInterrupt:
            print("\nProgram Terminated.")
            break
        except Exception as e:
            print("Error:", e)
            utime.sleep_ms(200)

if __name__ == "__main__":
    main()
```
You said

after step 2 where the code is generated

Gemini said

In **Step 2**, the code is **not generated as a separate file** automatically. Instead, Spyder outputs text-based decision rules in the **Spyder Console window** at the bottom-right of your screen.

You manually translate those printed rules into if / else statements inside the **Step 3** code running in **Thonny** on your RT1064 board.

Step-by-Step Location & Action

1. Locate the Output in Spyder

After running the Step 2 script in Spyder, look at the **IPython Console** (bottom-right panel). You will see text printed out that looks like this:

Plaintext

```
--- TRAINED AI RULES ---
|--- magnitude <= 110.00
|   |--- class: STATIONARY
|--- magnitude > 110.00
|   |--- magnitude <= 320.00
|   |   |--- class: WALKING
|   |--- magnitude > 320.00
|   |   |--- class: RUNNING
```

2. Open the Step 3 Script in Thonny

In **Thonny**, open the main program for your RT1064 board and find the predict_activity_ai function:

Python

```
# ============================================================
# EMBEDDED AI INFERENCE ENGINE
# ============================================================
def predict_activity_ai(magnitude, heading):
    """
    Offline Decision Tree model exported directly from Spyder.
    """
    if magnitude <= 110.0:
        return "STATIONARY"
    else:
```

```
    if magnitude <= 320.0:
        return "WALKING"
    else:
        return "RUNNING"
```

3. Update the MicroPython Code

Match the condition values (110.0, 320.0, etc.) inside predict_activity_ai() in **Thonny** with the exact numbers printed in your **Spyder console**.

Once updated, save the file in Thonny and click **Run**. The RT1064 EVK will now execute the trained AI rules completely offline.